

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY

Sector-128, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING
AND INFORMATION TECHNOLOGY

Placement Cell Automation System

Submitted To:

Ms. Kashish Mahajan

Department of Computer Science & Engineering and Information Technology

Submitted By:

Enrolment Number	Name of Student
992401040077	Kishan Soni
992401040073	Krishna Chawla
992401040079	Satyam Pundhir

Course: Software Engineering (24B11CS223) | B.Tech IT | 2025–2026



Acknowledgement

We sincerely thank Ms. Kashish Mahajan for her valuable guidance and motivation throughout the project. We are grateful to the open-source community for the tools and frameworks that made this project feasible. Software Engineering concepts studied during the course played a significant role in system design. We also thank our classmates, the Institute, and all those who supported the completion of this project.



Table of Contents



1. Introduction

1.1 Overview of Placement Cell Automation System

The Placement Cell Automation System (PCAS) is a comprehensive web-based platform designed to digitize and automate placement activities within an educational institution. It provides a centralized, role-based digital platform integrating student registration, academic profile management, company onboarding, job posting, eligibility filtering, and placement tracking.

Three primary stakeholders interact with the system:

- Students – register, maintain profiles, and apply to eligible job drives
- Placement Administrators – manage companies, schedule drives, and generate reports
- Recruiters – post job roles, shortlist candidates, and update recruitment status

1.2 Scope of the Project

- Student Profile Management: Registration, academic records, resume upload, eligibility verification
- Recruiter and Company Management: Company onboarding, job role posting, and access control
- Automated Eligibility Filtering: CGPA-based shortlisting of students for specific job roles
- Drive Scheduling and Coordination: Management of recruitment drive calendars and notifications
- Real-Time Status Tracking: Live updates on application status for students and recruiters
- Placement Reports: Generation of branch-wise and year-wise placement statistics
- Role-Based Access Control: Differentiated access for Students, Administrators, and Recruiters



2. Software Engineering Principles & Process Models

2.1 Software Engineering Principles

PCAS development was guided by Modularity (six independent modules), Reusability (shared UI and backend components), Abstraction (clean RESTful APIs), Maintainability (separated frontend/backend), and Separation of Concerns (authentication, logic, data, presentation in dedicated layers).

2.2 Agile Model & Sprint Planning

After evaluating Waterfall, Incremental, and Spiral models, Agile/Scrum was selected for PCAS due to requirement volatility, incremental delivery needs, and a small collaborative team. Nine sprints were completed covering requirements, design, authentication, profile management, job posting, eligibility filtering, drive scheduling, reporting, and final testing.

2.3 Project Planning – CPM, PERT & Gantt

Critical path activities included database schema design, user authentication, eligibility filtering engine, API integration, and integration testing. PERT three-point estimation was applied for uncertain activities:

Activity	O (days)	M (days)	P (days)	Te (days)	Owner
Database Schema Design	2	3	5	3.2	Krishna
Authentication Implementation	3	5	8	5.2	Krishna
Eligibility Filtering Engine	4	6	10	6.3	Satyam
API Integration	2	4	7	4.2	All
Integration Testing	3	5	9	5.3	All



3. Requirement Engineering

3.1 Functional Requirements

- FR-01: System shall allow students to register with valid institutional email and CGPA
- FR-02: System shall authenticate users with role-based access (Student, Admin, Recruiter)
- FR-03: Eligibility engine shall filter students based on CGPA threshold and eligible branches
- FR-04: Admins shall create and manage placement drives with date, company, and criteria
- FR-05: Recruiters shall view shortlisted applicants and update placement status
- FR-06: System shall generate branch-wise and year-wise placement reports
- FR-07: Students shall be notified upon application submission and status changes

3.2 Non-Functional Requirements

- Performance: System shall respond to API requests within 500ms under normal load
- Security: Passwords stored as bcrypt hashes; all routes protected with JWT middleware
- Scalability: Three-tier architecture supports horizontal scaling of API and DB layers
- Usability: Responsive UI operable on desktop and mobile browsers
- Reliability: MySQL with connection pooling ensures 99.5% uptime under normal usage

3.3 Software Requirement Specification (IEEE SRS)

The SRS document follows IEEE Std 830-1998 structure. Requirement validation was performed through stakeholder review, prototype walkthroughs, and cross-checking against functional scenarios. All requirements were traced to corresponding test cases to ensure completeness.



4. Software Design

4.1 UML Diagrams

4.1.1 Use Case Diagram

The Use Case Diagram below shows all actors (Student, Admin, Recruiter) and their interactions with the PCAS system boundaries.

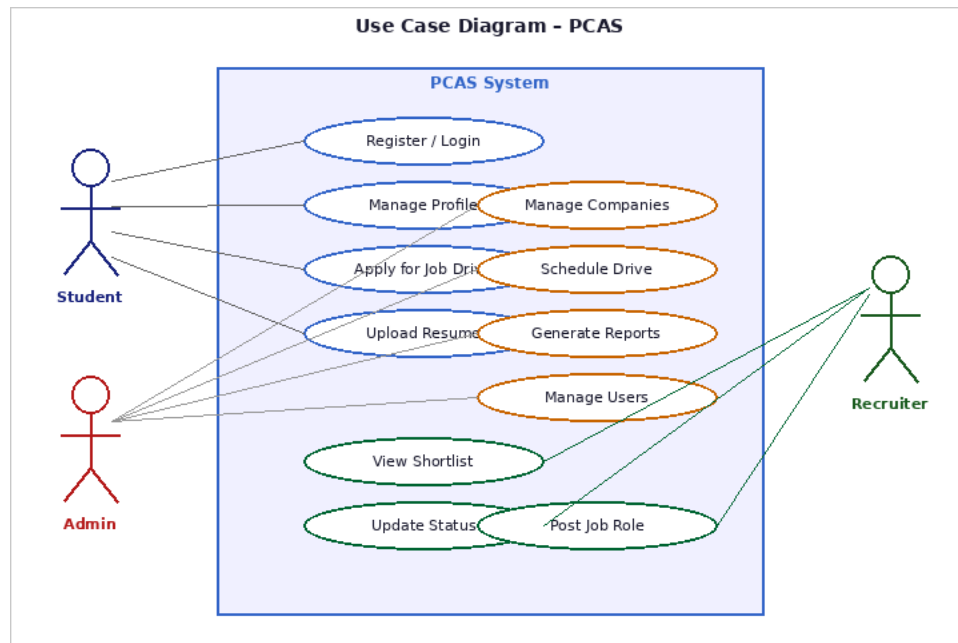


Figure 1: Use Case Diagram – PCAS

4.1.2 Activity Diagram

The Activity Diagram illustrates the control flow of the job application process including eligibility checking, application submission, and notifications.

Activity Diagram - Job Application Flow

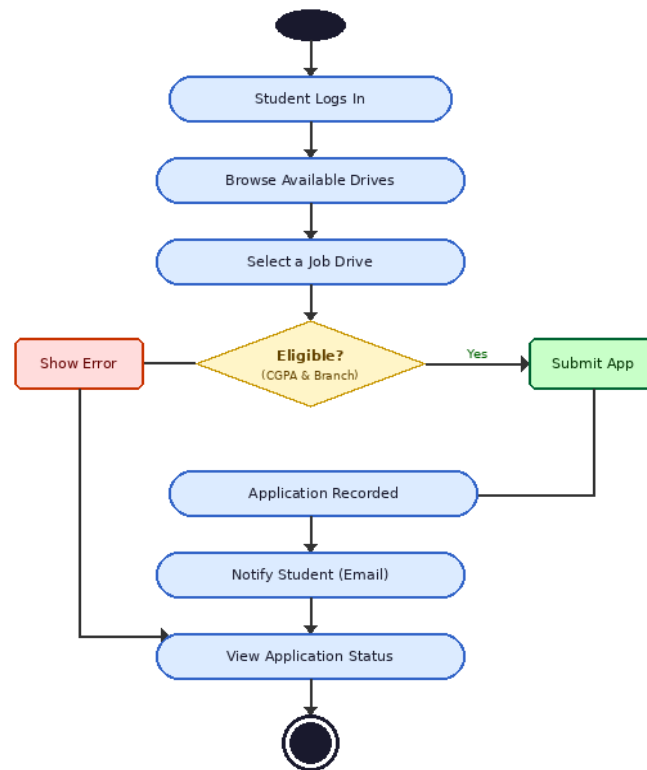


Figure 2: Activity Diagram – Job Application Flow

4.1.3 Class Diagram

The Class Diagram shows the six core entity classes – User, Student, Company, JobDrive, Application, and EligibilityEngine – along with their attributes, methods, and relationships.

Class Diagram - PCAS Core Entities

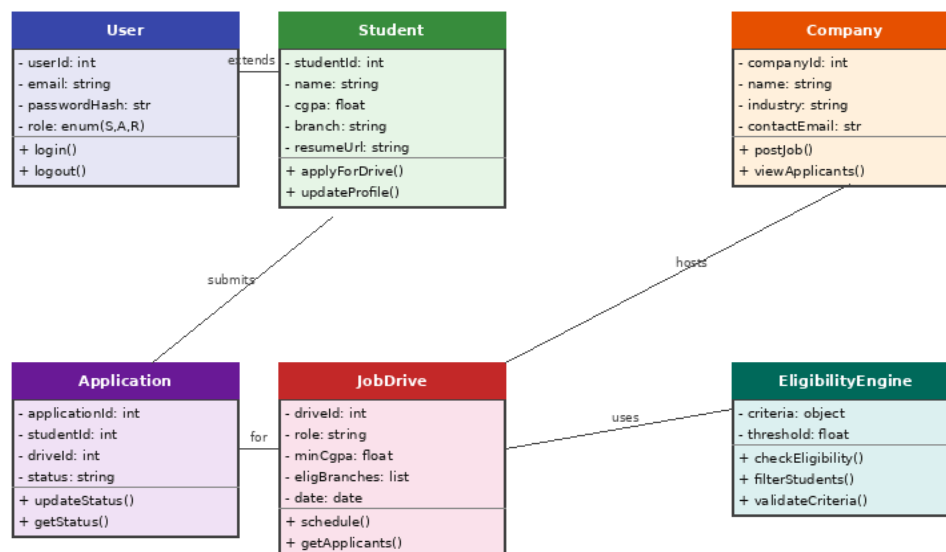


Figure 3: Class Diagram – PCAS Core Entities



4.1.4 Sequence Diagram

The Sequence Diagram captures the message flow between the Student, Frontend Browser, Express API Server, and AuthService/DB for login and job application operations.

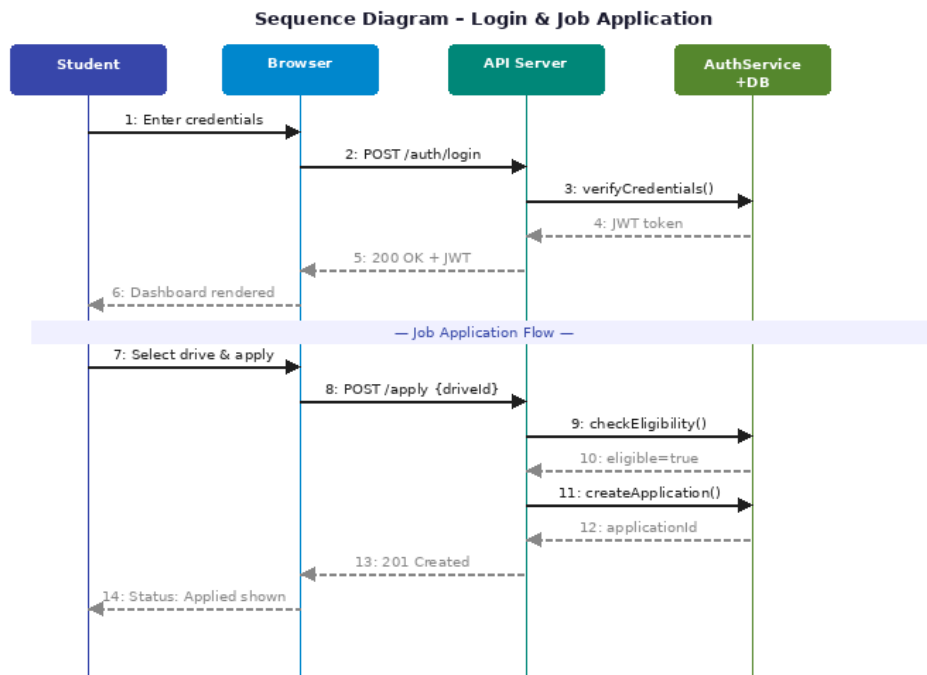


Figure 4: Sequence Diagram – Login & Job Application

4.2 Cohesion and Coupling

Cohesion (Functional Cohesion)

PCAS achieves functional cohesion — each module performs a single, well-defined responsibility:

- AuthService: Handles only authentication, password hashing, and JWT generation/validation
- StudentService: Handles only student profile creation, updates, and retrieval
- JobService: Handles only job posting creation, retrieval, and closure
- EligibilityEngine: Handles only CGPA and eligibility criteria computation (LCOM $\approx$ 0)
- ReportService: Handles only placement data aggregation and report generation
- NotificationService: Handles only sending alerts and messages to users

Object-Oriented Metrics: The EligibilityEngine class achieves LCOM close to 0, indicating high cohesion. Maximum Depth of Inheritance (DIT) = 1, keeping class structures simple and maintainable.

Coupling (Loose Coupling)

The system maintains loose coupling through:

- Data Coupling: Modules share only minimal data via typed function parameters and JSON API responses — no shared global state
- REST API Boundaries: Frontend and backend are completely decoupled, communicating only through well-defined HTTP endpoints
- Dependency Injection: Services injected as dependencies rather than instantiated inside controllers, enabling easy unit testing
- CBO Metric: ApplicationController coupled only to StudentService and JobService — not to ReportService or NotificationService, maintaining low CBO values

4.3 System Architecture & MVC



PCAS follows MVC architecture: Model (MySQL + Sequelize — Student, Company, Job, Application entities), View (HTML/CSS/JS — Dashboard, Profile, Jobs UI), Controller (Express.js route handlers — routes requests, invokes services, returns JSON). Communication uses HTTPS between client and server, and TCP/IP between server and database.



5. Software Construction

5.1 Coding Standards, Refactoring & Optimization

Standards adopted: camelCase for JS variables, PascalCase for classes, snake_case for DB columns, JSDoc comments on all API handlers, 2-space indentation, constants in environment variables.

Key refactoring: Eligibility logic extracted into EligibilityEngine service; repeated DB setup consolidated into db.js; multiple fetch queries unified into getStudentsByFilter(); report queries optimized with JOIN-based SQL replacing sequential queries.

Optimizations: DB indexing on student_id, cgpa, branch; connection pooling (pool size=10); server-side pagination on all list endpoints; placement report caching for 10 minutes; frontend asset minification.

5.2 Reusable Components

Frontend: DataTable (sortable, paginated), StatusBadge (Applied/Shortlisted/Placed/Rejected), ModalDialog (confirmations and alerts), FormValidator (centralized input validation). Backend: AuthService (JWT), EligibilityEngine (criteria checking), ReportGenerator (aggregation query builder), EmailNotifier (notification stub).

5.3 Defensive Programming

- Input Validation middleware validates all API request parameters before processing
- CGPA Boundary Check: values validated within [0.0, 10.0]
- SQL Injection Prevention: parameterized prepared statements throughout
- XSS Prevention: user fields sanitized using validator.js
- Resume Upload: PDF-only, max 2 MB enforced at middleware level



6. Software Metrics

6.1 Size-Oriented & Function Point Metrics

Metric	Value
Lines of Code (LOC)	~3,000 – 4,000
Frontend Pages	8 pages
Backend API Endpoints	18 endpoints across 6 route files
Database Tables	7 tables
Language Composition	JS: 72%, HTML: 15%, CSS: 10%, SQL: 3%
Reusable Components	8 (frontend: 4, backend: 4)
Total Function Points (UFP)	171 FPs

6.2 COCOMO Model

PCAS is categorized as Organic (small experienced team, well-understood domain, ~3,500 LOC). Basic COCOMO: Effort = $2.4 \times (3.5)^{1.05} \approx 9.0$ Person-Months; Development Time = $2.5 \times (9.0)^{0.38} \approx 6.5$ Months; Avg Team Size $\approx 2-3$ Developers. These estimates align with the actual 10-week, 3-member project.



7. Software Testing

7.1 Testing Strategies

White Box Testing (basis path, condition, loop testing) was applied to EligibilityEngine and AuthService. All 6 execution paths through login and eligibility were verified. All four CGPA × Branch combinations were tested. Black Box Testing covered registration, login, job application, drive creation, report generation, and status updates.

Boundary Value Analysis covered CGPA [0.0–10.0], password length [8–64 chars], resume size [1KB–2MB], CTC [1–100 LPA]. Equivalence Class Partitioning identified valid and invalid classes for all major inputs. Regression Testing was performed after each major feature addition.

7.2 Test Cases and Results

TC#	Test Case	Input	Expected Result	Status
TC-01	Valid Student Login	Registered email + correct password	Dashboard displayed, JWT issued	✓ Pass
TC-02	Invalid Password	Registered email + wrong password	401 Unauthorized error	✓ Pass
TC-03	Unregistered Email	Non-existent email	404 User not found	✓ Pass
TC-04	Student Registration	Valid name, email, CGPA, branch	Account created	✓ Pass
TC-05	Duplicate Email	Already registered email	409 Conflict error	✓ Pass
TC-06	Eligible Student Applies	CGPA 8.5, threshold 7.5	Application accepted	✓ Pass
TC-07	Ineligible CGPA	CGPA 6.8, threshold 7.5	Rejected – CGPA below requirement	✓ Pass
TC-08	Ineligible Branch	Branch ECE; eligible: CSE, IT	Rejected – Branch not eligible	✓ Pass
TC-09	Admin Creates Drive	Valid company, role, date, CGPA	Drive created, students notified	✓ Pass
TC-10	Past Date Drive	Drive date = yesterday	Validation error: invalid date	✓ Pass
TC-11	Recruiter Views Shortlist	Recruiter login + view applicants	Only eligible applicants listed	✓ Pass
TC-12	Status Update to Placed	Recruiter marks student Placed	Status updated, report reflects	✓ Pass
TC-13	Branch-wise Report	Admin generates CSE report	Correct placement count shown	✓ Pass
TC-14	PDF Resume Upload	Valid PDF < 2 MB	Resume uploaded and linked	✓ Pass
TC-15	Non-PDF Rejected	.docx file uploaded	Error: only PDF allowed	✓ Pass
TC-16	Oversized File	PDF > 2 MB	Error: file exceeds 2 MB limit	✓ Pass
TC-17	CGPA = 10.0 (max)	Student CGPA = 10.0	Accepted as valid value	✓ Pass
TC-18	CGPA = 10.1 (above max)	Student CGPA = 10.1	Validation error returned	✓ Pass
TC-19	Empty Required Fields	Submit form with blanks	Validation errors shown	✓ Pass
TC-20	Unauthorized Route	Student /admin/reports accesses	403 Forbidden response	✓ Pass



8. Conclusion

The Placement Cell Automation System (PCAS) successfully demonstrates how a well-engineered, modular web application can digitize and streamline institutional placement management. The system achieved all primary objectives: secure three-tier architecture with RBAC, automated CGPA-based eligibility filtering, real-time placement tracking, comprehensive analytics reporting, and robust security (bcrypt, JWT, input sanitization).

All key Software Engineering principles — modularity, separation of concerns, reusability, defensive programming, Agile development, MVC architecture, and comprehensive testing (white box, black box, BVA, ECP, regression) — were rigorously applied throughout the development lifecycle. The project provided hands-on experience in the complete SDLC, reinforcing theoretical Software Engineering concepts in a real-world applied context.



9. References

- IEEE Std 830-1998. Recommended Practice for Software Requirements Specifications. IEEE Computer Society.
- Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (9th ed.). McGraw-Hill Education.
- Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
- Booch, G., Rumbaugh, J., & Jacobson, I. (2005). The Unified Modeling Language User Guide (2nd ed.). Addison-Wesley.
- Myers, G. J., Sandler, C., & Badgett, T. (2011). The Art of Software Testing (3rd ed.). John Wiley & Sons.
- Node.js Official Documentation. (2024). <https://nodejs.org/en/docs>
- Express.js Documentation. (2024). <https://expressjs.com>
- MySQL Documentation. (2024). <https://dev.mysql.com/doc>
- Agile Alliance. (2024). Agile Manifesto and Principles. <https://agilemanifesto.org>
- OWASP Foundation. (2024). OWASP Top Ten. <https://owasp.org/www-project-top-ten>

